Devups

Tome II — Les données

Documentation du framework

Deuxième édition — novembre 2026 · Devups 7.0.0

Établie à partir du code du dépôt **spacekola/devups** (github.com/yuricoco/devups-package), version 7.0.0, et de sa suite de tests.

Sommaire

Chapitre 1 — Le datacore 4

1.1 Le principe 4
1.2 Le datacore d'un composant 4
1.3 Le datacore d'un module 4
1.4 Le datacore d'une entité 5
1.5 Les champs automatiques 6
1.6 Les trois choses que le datacore ne sait pas dire 6

Chapitre 2 — Les entités 7

2.1 La chaîne d'héritage 7
2.2 L'anatomie d'une entité 8
2.3 L'annotation de référence 9
2.4 Les propriétés `dv*` 9

Chapitre 3 — Les trois classes de la couche d'accès 11

3.1 L'architecture 11
3.2 Doctrine, et ce qu'il ne fait pas 11
3.3 La connexion partagée 11
3.4 Les erreurs SQL 11

Chapitre 4 — Lire 13

4.1 Les points d'entrée statiques 13
4.2 La clause `WHERE` 14
4.3 Tri, regroupement, limite 15
4.4 Les relations 15
4.5 Les méthodes terminales, et le piège `null` contre entité vide 16
4.6 Prévisualiser une requête sans l'exécuter 17
4.7 Exemples prêts à reprendre 17
4.8 Qualifier les colonnes 18

Chapitre 5 — Écrire 19

5.1 Les formes 19
5.2 Un piège d'écriture 19
5.3 Les transactions 19
5.4 Les accesseurs et le traitement de formulaire 20
5.5 Sérialisation 20

Chapitre 6 — Les migrations 21

6.1 Le principe 21
6.2 Les deux formats 21
6.3 La commande 22
6.4 Installation neuve contre base existante 22
6.5 Écrire une migration 22

Chapitre 7 — La traduction des données (`dvtranslate`) 23

7.1 Le modèle 23
7.2 La bonne séparation 23
7.3 Forcer une langue 23
7.4 Le défaut D15 — les lignes invisibles 24
7.5 Lire une colonne non traduite d'une entité traduite 24
7.6 Un second défaut, dans la même famille 25

Chapitre 8 — Pièges et vulnérabilités de la couche de données 26

Chapitre 1 — Le datacore

1.1 Le principe

Dans Devups, la structure des données est décrite par des fichiers JSON appelés **datacores**. Ils ne sont pas lus à l'exécution pour construire les requêtes : ils servent au générateur, au scan qui établit la liste des composants, modules et entités, au catalogue des droits (`rights:sync`) et à la documentation générée (`api:doc`, `doc:guide`).

La vérité d'exécution, elle, est dans les annotations des classes d'entité, lues par Doctrine. Le datacore est l'intention ; l'annotation est le contrat.

> **Corollaire.** Quand les deux divergent — et ils divergent, voir §1.6 —, c'est l'annotation qui gagne, parce que c'est elle que Doctrine lit pour construire la table. Le datacore ne redevient la vérité qu'au moment où l'on régénère l'entité, et il écrase alors tout ce qui a été ajouté à la main.

1.2 Le datacore d'un composant

`src/<composant>/<composant>Core.json` :

```
{
  "name": "monprojet",
  "template": "adminv2",
  "listmodule": []
}
```

Clé	Rôle
`name`	le nom du composant ; doit correspondre au nom du dossier
`template`	le gabarit de back-office
`listmodule`	ignoré — écrasé par le scan des sous-dossiers

La présence de ce fichier est la condition d'existence du composant : sans lui, le composant entier disparaît du framework.

1.3 Le datacore d'un module

`src/<composant>/<Module>/<module en minuscules>Core.json` :

```
{
  "name": "ModuleTask",
  "label": "Tâches",
  "icon": "fas fa-fw fa-check-square"
}
```

Clé	Effet
`name`	la clé de `module_configurations.php`
`label`	le libellé dans la barre latérale ; par défaut, le nom du module
`icon`	la classe d'icône ; par défaut `fas fa-fw fa-cog`

`{"name":"ModuleXxx"}` suffit à faire exister le module. Les autres clés (`id`, `project`, `nbent`, `listentity`) que l'on trouve dans les modules du socle sont inertes ou écrasées par le scan.

1.4 Le datacore d'une entité

`src/<composant>/<Module>/Core/<Entité>Core.json` :

```
{
  "name": "task",
  "label": "Tâche",
  "lang": ["name", "description"],
  "attribut": {
    "1": { "name": "name", "datatype": "string", "size": "255",
           "nullable": "not", "formtype": "text" }
  },
  "relation": [
    { "entity": "project", "cardinality": "manyToOne",
      "nullable": "default", "ondelete": "cascade" }
  ],
  "actions": [ { "code": "close", "label": "Clôturer" }, "archive" ],
  "methods": []
}
```

Les clés lues sont `name`, `label`, `attribut`, `relation`, `lang`, `actions` et `methods`. Toute autre clé est inerte.

Le bloc `attribut`

C'est un objet indexé par des chaînes numériques, pas un tableau.

Clé	Valeurs	Note
`name`	minuscules, chiffres et `_`	devient le nom de colonne
`datatype`	`string`, `text`, `integer`, `float`, `date`, `datetime`, `json`, `boolean`	devient `@Column(type=...)`
`size`	entier en chaîne	pertinent pour `string`
`nullable`	`"not"` (NOT NULL) ou `"default"` (nullable)	ignoré pour une relation (§1.6)
`formtype`	voir le Tome III	pilote le champ généré

`id` est implicite et ne doit jamais figurer dans `attribut` : le générateur l'ajoute avec `@Id @GeneratedValue`.

Le bloc `relation`

Clé	Valeurs
`entity`	le nom en minuscules de l'entité cible

Clé	Valeurs
`cardinality`	`manyToOne` , `oneToMany` , `manyToMany` , `oneToOne`
`nullable`	ignoré par le générateur (§1.6)
`ondelete`	`cascade` , `set null` , `restrict` — le seul attribut de `@JoinColumn` réellement interpolé

Le bloc `lang`

La liste des colonnes traduites. Sa présence fait de l'entité une entité traduite : le générateur produit en plus une classe `<Entité>_lang.php` et une table `<entité>_lang` (chapitre 7).

Le bloc `actions`

> **Nouveau en 7.0.** Toute entité a six actions implicites : `create` , `read` , `update` , `delete` , `export` , `import` . La clé `actions` déclare les actions métier qui s'y ajoutent. Chaque élément est soit une chaîne (`"archive"` , libellé déduit), soit un objet `{"code": ..., "label": ...}` . `php devups rights:sync` reporte ce catalogue dans la table `dvups_action` , où les rôles peuvent recevoir chaque action (Tome IV, chapitre 2).

1.5 Les champs automatiques

Toute entité mappée reçoit `created_at` , `updated_at` et `deleted_at` . Ils ne se déclarent pas dans `attribut` .

`deleted_at` est le support de la suppression douce : `$dvsoftdelete = true` fait que `__delete()` écrit une date au lieu de supprimer la ligne, et `QueryBuilder::handlesoftdelete()` filtre les lignes supprimées de toute lecture.

1.6 Les trois choses que le datacore ne sait pas dire

Besoin	Datacore	Réalité
contrainte d'unicité	aucune notion	`@UniqueConstraint(name=..., columns={...})` à la main
relation obligatoire	`"nullable":"not"` déclaré…	…jamais interpolé sur une relation ; la clé étrangère est NULLable dans tous les cas. `nullable=false` à ajouter à la main dans `@JoinColumn`
identité composite	aucune notion	`@Id @ManyToOne` à la main, pas de colonne `id` , `getId()` surchargée

Dans les trois cas, l'entité ne doit plus être régénérée : `core:g:crud` écrase l'annotation (Tome I, §6.7).

Chapitre 2 — Les entités

2.1 La chaîne d'héritage

```
stdClass
 └─ ModelSerializer      (dclass/devups/model/ModelSerializer.php)
     └─ Model            (dclass/devups/model/Model.php)
         │  use ModelTrait, DateTimeTrait
         └─ <Entité>     (src/<composant>/<Module>/Entity/)
```

Entre `Model` et l'entité, une classe de base réutilisable de `dclass/extends/` peut s'intercaler. En 7.0 :

Classe	Porte	Entités du socle qui en héritent
`UserCore`	les colonnes communes d'un utilisateur : identité, `email`, `phonenumber`, `password`, activation, réinitialisation, `lang`, jetons de session	`User`
`ImageCore`	une image : `reference`, `name`, `description`, `image`, `uploaddir`, dimensions	`Cmsresource`, `Tree_item_image`
`Dv_langCore`	une langue	`Dvups_lang`
`Dvups_migration`	une migration jouée	(table `dvups_migration`)
`Dvups_config_item`	déclarée traduite, sans table ni appelant	— (code mort)

Toutes portent l'annotation `@MappedSuperclass` : sans elle, Doctrine ignore les colonnes héritées et la table de l'entité enfant est créée incomplète.

> **`StatusTrait` est inutilisable.** Ce trait de `dclass/extends/` interroge une entité `Status_entity` qui n'existe plus dans le socle. Pour les statuts, utiliser l'API de `Status` (Tome IV, chapitre 6).

2.2 L'anatomie d'une entité

```php
<?php
/**
 * @Entity @Table(name="task")
 */
class Task extends Model implements JsonSerializable
{
    /**
     * @Id @GeneratedValue @Column(type="integer")
     * @var int
     */
    protected $id;

    /**
     * @Column(name="name", type="string", length=255)
     * @var string
     */
    protected $name;

    /**
     * @Column(name="description", type="text", nullable=true)
     * @var string
     */
    protected $description;

    /**
     * @ManyToOne(targetEntity="\Project")
     * @JoinColumn(onDelete="cascade")
     * @var \Project
     */
    public $project;

    public function __construct($id = null)
    {
        $this->id = $id;
        $this->project = new Project();
    }

    public function getId() { return $this->id; }

    public function jsonSerialize()
    {
        return ['id' => $this->id, 'name' => $this->name] + parent::jsonSerialize();
    }
}
```

Quatre conventions, toutes mesurées dans le code du framework :

1. Une colonne est `protected`, une relation est `public`. `ModelTrait::__get()` traite différemment une propriété déclarée et une propriété absente. Sur un attribut inconnu, il émet une notice puis arrête l'exécution (`die`) : l'écran entier disparaît. Deux exceptions : un attribut préfixé `_` (chargement paresseux d'une relation, ci-dessous) et une colonne traduite déclarée dans `$dvtranslated_columns`, qui renvoie `null` au lieu d'arrêter.

> **Le préfixe `_`.** `$task->_project` hydrate la relation à la demande et met le résultat en cache sur l'objet. C'est la seule forme d'attribut non déclaré que `__get()` accepte.

2. Le constructeur instancie les relations. `$this->project = new Project();` — sans cela, `$task->project->getName()` échoue sur `null`.

3. L'annotation doit toucher la classe. Le lecteur de Doctrine ne lit que le docblock immédiatement adjacent. Un docblock de prose intercalé fait échouer toute lecture de métadonnées.

4. `jsonSerialize()` ajoute, puis délègue. `+ parent::jsonSerialize()` récupère la sérialisation du socle, qui remonte `$this->dvserialised` et les relations déclarées dans `$this->with`.

2.3 L'annotation de référence

Annotation	Portée	Rôle
`@Entity`	classe	déclare une entité Doctrine
`@MappedSuperclass`	classe	déclare une classe de base dont les colonnes sont héritées
`@Table(name="...")`	classe	le nom de table
`@Table(..., uniqueConstraints={@UniqueConstraint(name=..., columns={...})})`	classe	contrainte unique nommée
`@Id`, `@GeneratedValue`	propriété	identité, auto-incrément
`@Column(name=, type=, length=, nullable=)`	propriété	une colonne
`@ManyToOne(targetEntity="\X")`	propriété	relation propriétaire
`@OneToMany(targetEntity="\X", mappedBy="y")`	propriété	relation inverse
`@ManyToMany(targetEntity="\X")`	propriété	relation n-n
`@JoinColumn(onDelete=, nullable=)`	propriété	la clé étrangère
`@var`	propriété	le type PHP, lu par l'hydratation

2.4 Les propriétés `dv*`

`ModelTrait` déclare des propriétés de contrôle sur toute entité. Les connaître évite de les prendre pour des colonnes.

Propriété	Défaut	Rôle
`$dvfetched`	`false`	l'entité a déjà été hydratée depuis la base
`$dvold`	`false`	une **copie** de l'état « avant », pour calculer le `SET` d'un `UPDATE`
`$dvsoftdelete`	`false`	si vrai, `__delete()` écrit `deleted_at` au lieu de supprimer
`$dvinrelation`	`false`	l'entité est chargée comme relation d'une autre
`$dvtranslate`	`false`	l'entité est traduite (chapitre 7)
`$dvid_lang`	`false`	la langue courante de l'instance
`$dvtranslated_columns`	`[]`	la liste des colonnes traduites
`$dvpersistence`	`[]`	les valeurs à écrire
`$dvserialised`	`[]`	les valeurs à sérialiser en JSON

Propriété	Défaut	Rôle
`$with` (protégée)	`[]`	les relations à inclure dans `jsonSerialize()`

Corrigé en 7.0 — `$dvold` est un vrai clone. `Model::hydrate()` posait autrefois `$this->dvold = $this`, une référence au même objet : modifier un champ modifiait aussi sa valeur « avant », aucune différence n'était détectée et l'`UPDATE` n'écrivait rien. `hydrate()` pose désormais `clone $this`, comme le faisait déjà `find()`. Hydrater depuis `new X($id)` ou depuis `X::find($id)` donne le même résultat à l'écriture.

Chapitre 3 — Les trois classes de la couche d'accès

3.1 L'architecture

```
Model           (dclass/devups/model/Model.php)
 └─ use ModelTrait
     → les points d'entrée STATIQUES : Entity::find(), ::where(), ::create()…
     → chaque point d'entrée fabrique un `new QueryBuilder($entity)`

QueryBuilder    (dclass/devups/Tchutte/QueryBuilder.php)   extends DBAL
     → construit le SQL (SELECT / INSERT / UPDATE / WHERE / JOIN / ORDER BY…)
     → API fluide : chaque méthode retourne $this

DBAL            (dclass/devups/Tchutte/DBAL.php)           extends Database
     → exécute le SQL et hydrate les entités
     → expose la connexion PDO partagée
     → lève DBALException sur toute erreur SQL
```

3.2 Doctrine, et ce qu'il ne fait pas

L'`EntityManager` de Doctrine, accessible par `DBAL::getEntityManager()`, sert à exactement deux choses : lire les métadonnées d'annotation (quelle table, quelles colonnes, quelles associations) et comparer ou créer le schéma (`schema:update`, `install`).

Aucune requête métier ne passe par Doctrine. Ne pas aller chercher une connexion dans `getEntityManager()` : c'est une connexion différente, volontairement.

3.3 La connexion partagée

```
$pdo = Model::pdo();
```

Retourne le même objet PDO que celui utilisé par `find()`, `__insert()`, `__update()` et tout le `QueryBuilder`. À utiliser pour toute requête SQL qui ne passe pas par le `QueryBuilder`. Jamais `new PDO(...)` : cela ouvrirait une connexion séparée, invisible à toute transaction ouverte par `Model::beginTransaction()`.

3.4 Les erreurs SQL

> **Nouveau en 7.0.** Toute erreur SQL lève une `dclass\devups\Tchutte\DBALException` (qui étend `RuntimeException`), avec la requête (`$e->sql`) et ses paramètres (`$e->param`). Hors production, le message inclut le SQL. Les versions 4.x affichaient l'erreur puis tuaient la requête : une violation de contrainte n'était pas rattrapable par un contrôleur. Elle l'est désormais.

```
use dclass\devups\Tchutte\DBALException;

try {
    Task::create(['name' => $name, 'project_id' => $projectId]);
} catch (DBALException $e) {
    Log::error($e);
    return ['success' => false, 'detail' => t('Impossible d\'enregistrer la tâche')];
}
```

Vérifier avant d'écrire reste la meilleure forme quand la condition est connue (§4.7) : un message métier clair vaut mieux qu'une exception SQL traduite.

Chapitre 4 — Lire

4.1 Les points d'entrée statiques

Toute entité hérite de `ModelTrait`, qui expose ces méthodes directement sur le nom de la classe.

Méthode	Rôle	Sur zéro résultat
`Entity::find($id, $id_lang = null)`	lecture par id (ou tableau d'ids, ou tableau de conditions)	entité **vide**, jamais `null`
`Entity::findOrNull($id, $id_lang = null)`	idem	`null`
`Entity::where($col, $op, $val, $id_lang = null)`	fabrique un `QueryBuilder` déjà filtré	(chaîner)
`Entity::whereId($id)`	`this.id = $id`, ou `IN` si tableau	—
`Entity::whereIn($col, $values)`	`IN (...)`	—
`Entity::first($id_lang)` / `::firstOrNull()`	le premier enregistrement	entité vide / `null`
`Entity::last()` / `::lastrow()`	le dernier	entité vide
`Entity::firstOrCreate($constraint, $data)`	lit, ou crée si absent	crée
`Entity::firstOrNew($constraint, $data)`	lit, ou instancie sans insérer	—
`Entity::count($col = null, $val = null)`	`COUNT(*)`, filtré ou non	`0`
`Entity::sum()`, `::avg()`, `::max()`	agrégats	—
`Entity::all($sort = 'id', $order = 'asc', $id_lang)`	tout	`[]`
`Entity::allrows(...)`	tout, en lignes brutes	`[]`
`Entity::trashed(...)`	les lignes en suppression douce	`[]`
`Entity::getbyattribut($attribut, $valeur)`	une entité, par n'importe quelle colonne	entité vide
`Entity::select($columns = '*', ...)`	un `QueryBuilder` avec projection	—
`Entity::initQb($defaultjoin = true)`	un `QueryBuilder` vierge	—
`Entity::with($relation, $colonnes)`	chargement de relation	—
`Entity::orderBy($col, $sort)`, `::distinct($cols)`	tri, distinct	—
`Entity::lazyloading($order = "")`	listing paginé de l'API (Tome III)	—
`Entity::truncate()`	vide la table	—
`Entity::dclone($id, $update)`	duplique une ligne	—

`Entity::where(...)` est le point d'entrée du quotidien : il équivaut à `Entity::initQb()->where(...)`.

`$defaultjoin = false` désactive les `LEFT JOIN` automatiques des relations `manyToOne` — utile quand la requête n'a besoin d'aucune relation.

4.2 La clause `WHERE`

Comparaisons

```
->where('this.status', '=', 'active')          // opérateur explicite
->where('this.id', $id)                         // deux arguments = égalité
->where('this.parent_id', null)                 // deux arguments, null = IS NULL
->where(['status' => 'active', 'type' => 'X'])  // tableau = égalités en AND
->andwhere($col, $op, $val)
->orWhere($col, $op, $val)
->whereId($id)
->where($unProject)                             // project.id = :project_id
```

Le préfixe `this.` désigne l'entité de base. Sans préfixe, la colonne devient ambiguë dès qu'une jointure est en jeu (§4.8).

Les opérateurs reconnus

```
=  <  >  <=  >=  <>  !=  <=>
like  'like binary'  'not like'  ilike  'not ilike'  between  is
&  |  ^  <<  >>   rlike  regexp  'not regexp'
~  ~*  !~  !~*  ~~*  !~~*   'similar to'  'not similar to'
```

> **Corrigé en 7.0.** La forme à deux arguments est normalisée : `where('col', $valeur)` est une égalité, `where('col', null)` produit `IS NULL`. Auparavant, une valeur qui n'était pas un opérateur connu pouvait être liée comme opérateur et donner un résultat faux sans erreur.

Recherche « contient »

```
->whereLike('label', $q)         // LIKE '%q%', en AND
->whereLike('label', $q, 'l')    // LIKE 'q%' (préfixe)
->whereLike('label', $q, 'r')    // LIKE '%q' (suffixe)
->orWhereLike('city', $q)        // en OR
->where('col')->like($q)         // forme chaînée
```

Ces méthodes sont paramétrées : la valeur n'entre jamais dans le texte SQL.

Groupe de conditions en OR

```
Project::where('this.status', '=', 'active')
    ->whereGroup(function ($qb) use ($q) {
        $qb->whereLike('name', $q)
           ->orWhereLike('code', $q)
           ->orWhereLike('description', $q);
    })
    ->orderBy('this.name')
    ->get();
// WHERE ... AND status = :x AND (name LIKE :a OR code LIKE :b OR description LIKE :c)
```

Un second argument `$outerLink` (`'AND'` par défaut) rattache le groupe entier en OU au reste.

À ne pas confondre avec `whereRaw($constraint, $link)` et `where_str($constraint, $link)` — deux méthodes au corps identique — qui prennent une chaîne SQL brute, jamais paramétrée. Dernier recours, jamais avec une valeur venant d'un utilisateur.

Listes, nulls, dates, intervalles

```
->whereIn('this.id', [1, 2, 3])
->whereIn('this.status', ['a', 'b'], 'NOT')
->where('col')->in([1, 2, 3])        ->where('col')->notIn($values)
->whereNull('col')                   ->whereNotNull('col')
->whereBetween('col', [$min, $max])  ->where('col')->between($v1, $v2)
->whereDate('col', $v)  ->whereDay(...)  ->whereMonth(...)  ->whereYear(...)  ->whereHour(...)
```

> **Non corrigé : `whereIn()` et `whereBetween()` ne sont pas paramétrés.** `whereIn()` entoure ses chaînes de quotes par `qb_sanitize()` sans les échapper, et `whereBetween()` interpole ses bornes directement. Toute valeur venant d'une requête HTTP doit être castée (`(int)`) ou validée (liste blanche) avant d'y entrer — ou passer par `whereGroup()` et des `where('col', '=', $v)` répétés.

4.3 Tri, regroupement, limite

```
->orderBy('this.name')                // ASC implicite
->orderBy('this.name', 'DESC')
->orderBy('col1')->orderBy('col2')    // cumulables
->rand()                              // ORDER BY RAND()
->groupBy('this.project_id')
->limit(10)                           // LIMIT 10
->limit(0, 10)                        // LIMIT 0, 10
->limit(-5)                           // les 5 DERNIERS
->distinct('col')
```

`orderBy('project.position')` — une colonne d'une relation jointe — fonctionne si la relation est jointe par défaut.

4.4 Les relations

Chaque `@ManyToOne` de l'entité de base est automatiquement jointe en `LEFT JOIN`, avec pour alias le nom de la table cible :

```
LEFT JOIN `project` ON `project`.id = task.project_id
```

> **Ce `LEFT JOIN` ne charge que l'`id`.** Accéder à `$task->project->name` sans rien de plus renvoie une chaîne vide — pas une erreur, rien de visible.

```
// SANS with() : $row->project existe, ses colonnes hors id sont vides
$rows = Task::where('this.done', '=', 0)->get();

// AVEC with() : les colonnes demandées sont réellement hydratées
$rows = Task::where('this.done', '=', 0)
    ->with('project', ['name', 'description'])
    ->get();
echo $rows[0]->project->name;
```

`with(string $relation, array $columns = [])` prend le nom de la **propriété** de relation (`public $project;` → `'project'`).

Méthode	Effet
`with($relation, $columns)`	hydrate réellement les colonnes de la relation
`without(...$relations)`	exclut des relations du `LEFT JOIN` par défaut
`initQb(false)`	désactive tous les `LEFT JOIN` par défaut
`join($classname, $on)`	jointure explicite
`leftjoin(...)`, `leftJoinOn($classname, $alias, $constraint)`	jointures à gauche
`innerjoin(...)`, `innerJoinOn(...)`	jointures internes
`join_str($classname, $on)`	jointure en SQL brut

`with()` est aussi pilotable depuis la requête HTTP : l'en-tête `Columns` portant `{"project":["name","description"]}` est converti en appels `with()`.

4.5 Les méthodes terminales, et le piège `null` contre entité vide

C'est le piège le plus coûteux de cette couche.

Méthode	Zéro résultat
`->get($column = "*")`	`[]`
`->getInstance()` / `->first()`	entité **vide** (`new X`), jamais `null`
`->firstOrNull()` / `->lastOrNull()`	`null`
`Entity::find($id)`	entité vide
`Entity::findOrNull($id)`	`null`
`->getRows($column, $callback)`	tableau de lignes brutes, sans hydratation
`->row()`	une ligne brute
`->getValue()` / `->getValues()`	valeur(s) scalaire(s)
`->count()`, `->sum()`, `->avg()`, `->max()`, `->min()`	scalaire
`->cursor($callback)`	parcours ligne à ligne

Après un `find()`, `getInstance()` ou `first()` sans le suffixe `OrNull`, ne jamais tester `if ($entity)` seul : un objet PHP est toujours vrai.

```
// FAUX — ne détecte jamais une entité vide
$existing = Project::where('this.code', '=', $code)->getInstance();
if ($existing) { /* toujours vrai */ }

// CORRECT
$existing = Project::where('this.code', '=', $code)->firstOrNull();
if ($existing && $existing->getId()) { /* vraiment trouvé */ }
```

4.6 Prévisualiser une requête sans l'exécuter

```
QueryBuilder::$debug = true;
$result = Project::where('this.status', '=', 'active')->get();
// $result vaut ['query' => ..., 'sql' => ..., 'parameters' => [...]]
QueryBuilder::$debug = false;   // à ne pas oublier : la propriété est statique
```

`->getSqlQuery()` donne le SQL d'une requête sans passer par le mode global. Utile pour prouver qu'une valeur est bien liée et non interpolée : une valeur contenant une apostrophe reste dans `parameters`, jamais dans le texte SQL.

4.7 Exemples prêts à reprendre

Lecture par id avec « non trouvé » explicite

```
$project = Project::findOrNull($id);
if (!$project) {
    return ['success' => false, 'detail' => t('Projet introuvable')];
}
```

Jointure avec chargement réel des colonnes

```
$tasks = Task::where('this.project_id', '=', $id)
    ->where('this.done', '=', 0)
    ->with('project', ['name', 'code'])
    ->orderBy('this.created_at', 'DESC')
    ->get();
```

Recherche multi-champs en OU, réponse au contrat figé

```
$qb = Project::where('this.status', '=', 'active');
if (($q = trim($q)) !== '') {
    $qb->whereGroup(fn ($sub) => $sub->whereLike('name', $q)->orWhereLike('code', $q));
}
$projects = $qb->orderBy('this.name')->get();

// Mise en forme explicite, jamais un jsonSerialize() qui exposerait toutes les colonnes
return ['success' => true, 'projects' => array_map(fn ($p) => [
    'id' => $p->getId(), 'code' => $p->code, 'name' => $p->name,
], $projects)];
```

Vérifier l'existence avant d'écrire

```
$existing = Project::where('this.code', '=', $code)->firstOrNull();
if ($existing && $existing->getId()) {
    return ['success' => false, 'detail' => t('Code déjà utilisé')];
}
```

4.8 Qualifier les colonnes

Quand une entité est traduite, la jointure de traduction ramène la table `dvups_lang` sous l'alias `l`, qui a sa propre colonne `name`. Sur une entité traduite qui a aussi une colonne `name`, un `WHERE name = ...` non qualifié devient :

```
SQLSTATE[23000] ... Column 'name' in where clause is ambiguous
```

> **Règle de travail.** Toujours qualifier : `this.<colonne>`. Sur une entité non traduite aussi, par habitude : cela ne coûte rien et prévient le jour où l'entité devient traduite.

Chapitre 5 — Écrire

5.1 Les formes

```
// création
$id  = Task::create(['name' => 'x', 'project_id' => 3]);       // un id
$ids = Task::create(['name' => 'a'], ['name' => 'b']);          // plusieurs lignes
$t   = Task::createInstance([...]);                              // l'instance

// mise à jour
$task = Task::find($id);
$task->name = 'nouveau nom';    // __set magique, même sur une propriété protected
$task->__update();              // UPDATE des colonnes modifiées
$task->__update(['name' => 'x']);   // UPDATE ciblé
$task->__save();                // insert ou update selon la présence d'un id

// bas niveau
$task->__insert();
$task->__delete();              // DELETE, ou suppression douce si dvsoftdelete
$task->__dclone($update);       // duplique la ligne

// statiques
Task::update($valeurs, $clé, $where, $classname);
Task::delete($id, $force = false);
```

Il n'y a pas de `setXxx()` fluide généré : l'affectation directe est le mécanisme. Un `setXxx()` écrit à la main a un autre rôle : il est appelé par le traitement de formulaire (§5.4).

5.2 Un piège d'écriture

`Entity::create()` attend le nom de colonne physique pour une relation.

```
Task::create(['project_id' => 3]);   // correct
Task::create(['project' => 3]);      // échoue : "Unknown column 'project'"
```

La forme `'project.id' => 3` fonctionne dans la charge utile d'API (Tome III), pas dans `create()`.

5.3 Les transactions

```
Model::beginTransaction();   // niveau 0 : vraie transaction PDO ; niveau > 0 : SAVEPOINT
try {
    $task->__insert();
    $history->__insert();
    Model::commit();          // niveau 0 : COMMIT réel ; niveau > 0 : libère le SAVEPOINT
} catch (\Throwable $e) {
    Model::rollBack();        // niveau 0 : ROLLBACK ; niveau > 0 : ROLLBACK TO SAVEPOINT
    throw $e;
}
```

La forme recommandée protège contre un `rollBack()` oublié sur un retour anticipé :

```
$taskId = Model::transaction(function () use ($data) {
    $taskId = Task::create($data);
    Task_history::create(['task_id' => $taskId, 'event' => 'created']);
    return $taskId;
});
```

L'imbrication est transparente : deux méthodes qui appellent chacune `Model::transaction()` peuvent s'appeler l'une l'autre sans lever le « There is already an active transaction » de PDO. `Model::transactionDepth()` donne le niveau courant.

> **Contrepartie de la connexion unique.** Une transaction laissée ouverte empoisonne tout ce qui suit dans la même requête HTTP. Toujours `Model::transaction()`, ou un `try/catch` avec `rollBack()` garanti.

5.4 Les accesseurs et le traitement de formulaire

Un `setXxx()` écrit à la main est un point de validation appelé par `Controller::formWithPost()`. Son contrat de retour est particulier : il n'y a aucun `try/catch` autour, et sa valeur de retour, si elle n'est pas vide, est traitée comme un message d'erreur.

```
public function setCode($code)
{
    $code = strtoupper(trim($code));
    if (!preg_match('/^[A-Z0-9_]{2,20}$/', $code)) {
        return t('Le code doit faire 2 à 20 caractères : lettres, chiffres, _');
    }
    $this->code = $code;
    return null;   // pas d'erreur
}
```

Un setter de validation retourne une chaîne d'erreur, il ne lève pas d'exception. Conséquence agréable : exposer un nouveau champ validé ne demande aucune modification du contrôleur.

5.5 Sérialisation

`ModelSerializer::jsonSerialize()` remonte `$this->dvserialised` plus, pour chaque clé de `$this->with`, la sérialisation de la relation correspondante. Une entité surcharge en ajoutant, puis en déléguant (§2.2).

> **Pour une réponse d'API, préférer une mise en forme explicite.** Sérialiser l'entité expose toutes ses colonnes, y compris celles qui n'ont jamais été documentées comme publiques, et fige le contrat de réponse sur la structure de la table. `User` montre la forme à suivre : `restrictToPublicProfile()` retire l'e-mail et le téléphone quand le demandeur n'est pas le propriétaire (Tome III, §8.6).

Chapitre 6 — Les migrations

> **Nouveau en 7.0.** Les versions 4.x faisaient évoluer le schéma par `schema:update --force` sur chaque environnement, ou par des fichiers SQL lancés à la main. La 7.0 apporte un vrai système de migrations, versionnées et tracées.

6.1 Le principe

Une migration est un fichier de `database/migration/`. `php devups migrate` joue, dans l'ordre alphabétique des noms de fichier, celles qui ne l'ont pas encore été sur la base courante, et enregistre chacune (nom, lot, date) dans la table `dvups_migration`.

Nommer les fichiers par date garantit l'ordre :

```
database/migration/
├── 2026_11_07_dvups_log_schema.sql
├── 2026_11_08_phase4_cms_rename.sql
├── 2026_11_08_phase4_drop_old_images.php
└── 2026_11_20_create_task.sql
```

6.2 Les deux formats

SQL brut, une ou plusieurs instructions :

```
CREATE TABLE task (
  id INT AUTO_INCREMENT NOT NULL,
  project_id INT DEFAULT NULL,
  name VARCHAR(255) NOT NULL,
  ...
  PRIMARY KEY(id)
) DEFAULT CHARACTER SET utf8mb4;
```

PHP, quand la migration doit décider ou vérifier avant d'agir. Le fichier retourne un objet avec une méthode `up(PDO $pdo)` :

```
<?php
return new class {
    public function up(\PDO $pdo) {
        $count = (int)$pdo->query("SELECT COUNT(*) FROM task WHERE code IS NULL")-
>fetchColumn();
        if ($count > 0) {
            throw new \RuntimeException("$count tâches sans code : reprise à faire avant la
contrainte.");
        }
        $pdo->exec("ALTER TABLE task ADD UNIQUE INDEX uniq_task_code (code)");
    }
};
```

Une exception arrête le lot proprement : la migration est signalée `[ECHEC]` et n'est pas marquée comme jouée. C'est la forme à utiliser pour toute suppression de données qui dépend d'une reprise

préalable : la migration vérifie elle-même que la reprise est complète, et refuse sinon, sans rien supprimer.

6.3 La commande

```
php devups migrate:status        # jouées et en attente
php devups migrate --dry-run     # affiche le SQL sans l'exécuter
php devups migrate               # joue les migrations en attente, en un seul lot
php devups migrate --step        # un lot par migration
php devups migrate --force       # saute la confirmation des migrations destructives
```

Une migration qui contient `DROP`, `TRUNCATE` ou `DELETE` est marquée **destructive** : elle demande une confirmation explicite (taper `oui`) ou `--force`. En production (`APP_PROD=true`), seul `--force` passe, jamais la confirmation interactive.

> **Avant une migration destructive, une sauvegarde.** `mysqldump` complet, hors du dépôt, vérifié lisible. Le framework demande la confirmation ; il ne fait pas la sauvegarde à votre place.

6.4 Installation neuve contre base existante

`php devups install` construit le schéma complet depuis les entités actuelles. Les migrations présentes à ce moment sont donc **marquées jouées sans être exécutées** (« base de départ ») : les rejouer échouerait sur des tables déjà là. Seule une base **antérieure** à une migration doit la jouer, via `migrate`.

6.5 Écrire une migration

1. Modifier l'entité (annotation).
2. `php devups schema:update --dump-sql` : Doctrine propose le SQL qui aligne la base sur l'entité.
3. Copier ce SQL dans un nouveau fichier daté de `database/migration/`, le relire, l'ajuster (reprise de données, ordre des `DROP` quand des clés étrangères lient les tables entre elles).
4. `php devups migrate --dry-run`, puis `php devups migrate`.
5. `schema:update --dump-sql` ne doit plus rien proposer.
6. Committer l'entité et la migration ensemble.

Le moteur est `dclass/devups/Migration/MigrationRunner.php`, couvert par `tests/Integration/MigrationRunnerTest.php` : ordre, idempotence, dry-run, refus d'une migration destructive, `--force`, `--step`, format PHP, base de départ.

Chapitre 7 — La traduction des données (`dvtranslate`)

Ce chapitre traite des **données** traduites, stockées en base (le libellé d'une catégorie en français et en anglais). La traduction de l'**interface** (`t()`, fichiers `lang/*.php`) est au Tome III, chapitre 7.

7.1 Le modèle

Une entité traduite déclare :

```
public $dvtranslate = true;
public $dvtranslated_columns = ["label"];
```

Son schéma se dédouble :

Table	Contenu
`<entité>`	les colonnes non traduites, dont la clé machine
`<entité>_lang`	une ligne par langue : les colonnes traduites et `lang_id`
`dvups_lang`	les langues (`id`, `iso_code`, `active`, `name`…)

Entités traduites du socle en 7.0 : `Tree`, `Tree_item`, `Status`, `Cmscontent`, `Notificationtype`, `Reportingmodel`, `Country`, `Dvups_entity`.

7.2 La bonne séparation

Le code se réfère à une **clé machine** non traduite, jamais au libellé : `tree.name` (et non `tree_lang.label`), `tree_item.slug` (et non `tree_item_lang.name`), `status._key`. Un libellé change quand un traducteur le change ; une clé machine ne doit jamais changer.

7.3 Forcer une langue

Appel	Effet
`->setLang($id_lang)`	fixe la langue du builder
`Entity::find($id, $id_lang)`	
`->firstOrNull($id_lang)` / `->first($id_lang)`	
en-tête HTTP `Dlang` / paramètre `?dlang=fr`	force la langue d'une réponse d'API

Sans langue forcée, les colonnes traduites reviennent concaténées sur toutes les langues par un `GROUP_CONCAT`, et le consommateur les sépare — c'est ce qui produit l'objet `{fr: ..., en: ...}` qu'une application cliente reçoit.

7.4 Le défaut D15 — les lignes invisibles

C'est le défaut le plus sérieux de cette couche, et il touche toute entité traduite.

`QueryBuilder::initSelect()` — appelé par toute lecture ORM — construit, quand aucune langue n'est forcée :

```
left join <table>_lang on (<table>_lang.<table>_id = `<table>`.id)
join dvups_lang l on (<table>_lang.lang_id = l.id)     -- INNER JOIN, pas LEFT
```

Si la ligne de base n'a **aucune** ligne `<table>_lang`, le `LEFT JOIN` produit des colonnes `NULL`, et le `JOIN` suivant sur `NULL = l.id` élimine la ligne entière — y compris pour un `WHERE this.id = X`. La ligne existe en base, n'est pas perdue, et devient invisible à l'ORM, silencieusement.

Avec une langue forcée, la branche est différente : le filtre de langue est dans la clause `ON` du `LEFT JOIN`, sans `JOIN dvups_lang`. La ligne reste visible ; la colonne traduite revient `NULL`.

Contournement : forcer la langue. La racine n'est pas corrigée : remplacer le `JOIN` par un `LEFT JOIN` changerait le résultat de chaque lecture traduite du framework, dans tout projet. C'est un arbitrage de l'auteur du socle.

Côté écriture, mettre à jour une entité traduite sans ligne `_lang` produirait une erreur SQL illisible. `Model::__update()` refuse donc explicitement, avec un motif clair, avant d'y arriver.

L'outil de diagnostic

`DvtranslateInspector` (`dclass/devups/model/DvtranslateInspector.php`, lecture seule) :

Méthode	Rôle
`invisibleRows($entityClass)`	compte et liste les lignes invisibles d'une entité
`isInvisible($entityClass, $id)`	cet id existe-t-il en base tout en étant invisible à l'ORM ?
`diagnose($entityClass, $id)`	distingue `no_translation`, `orphan_lang`, `visible`, `not_found`, `not_translated`, `schema_missing`

> **La règle qui évite le défaut.** Toute création d'une entité traduite doit créer sa ligne de traduction dans chaque langue. `__insert()` le fait (une ligne `_lang` par langue de `dvups_lang`) ; une insertion SQL directe — script, migration, import — doit le faire elle-même.

7.5 Lire une colonne non traduite d'une entité traduite

Si le besoin est de lire une clé machine ou un drapeau d'une entité traduite, et que la ligne peut ne pas avoir de traduction, passer par une requête préparée :

```
$stmt = Model::pdo()->prepare("SELECT branches FROM tree WHERE name = ?");
$stmt->execute([$treeName]);
$branches = $stmt->fetchColumn();
```

C'est ce que fait `Tree::buildCacheData()` (Tome IV).

7.6 Un second défaut, dans la même famille

`QueryBuilder::get($colonne, $id_lang)` appelle `initSelect()` **avant** de poser la langue. Passer `$id_lang` en second argument est donc sans effet : le libellé revient concaténé sur toutes les langues. La forme correcte :

```
$rows = Status::where('this.entity', '=', 'task')
    ->setLang($id_lang)
    ->get();
```

Chapitre 8 — Pièges et vulnérabilités de la couche de données

Récapitulatif, avec l'état réel de chacun en 7.0.

#	Objet	État	Détail
1	`whereLike()`, `like()` et variantes	corrigé	paramétrés
2	`whereIn()` sur des chaînes	**non corrigé**	quotes sans échappement ; valider ou caster avant
3	`whereBetween()`	**non corrigé**	bornes interpolées ; valider avant
4	forme à deux arguments de `where()`	corrigé	égalité, ou `IS NULL`
5	relations `manyToOne`	documenté	seul l'`id` est chargé ; `with()` au-delà
6	`getInstance()`, `first()`, `find()` sur zéro résultat	documenté	entité vide, jamais `null` ; `if ($x && $x->getId())`
7	`whereRaw()` / `where_str()`	documenté	SQL brut, jamais avec une donnée utilisateur
8	connexion PDO	corrigé	une seule connexion par processus
9	`$dvold` par référence	**corrigé en 7.0**	`hydrate()` pose un vrai clone
10	erreur SQL fatale	**corrigé en 7.0**	`DBALException`, rattrapable
11	D15 — `INNER JOIN dvups_lang`	**non corrigé**	ligne traduite sans traduction invisible ; forcer la langue
12	`get($col, $id_lang)`	**non corrigé**	second argument sans effet ; `setLang()` avant
13	colonne ambiguë sur une entité traduite	documenté	toujours `this.<colonne>`
14	`__get()` sur un attribut non déclaré	comportement du framework	arrêt de l'exécution ; la clé doit nommer une propriété réelle
15	`StatusTrait`	code mort	référence une entité qui n'existe plus

La règle pratique qui découle des points 2 et 3 : toute valeur venant d'une requête HTTP qui doit entrer dans un `whereIn()` ou un `whereBetween()` doit être validée ou castée avant. Le `QueryBuilder` ne le fait pas à votre place sur ces deux méthodes précises — contrairement à `where()`, `whereLike()` et `whereGroup()`.

Fin du Tome II. Le Tome III traite de l'interface : les routes, les contrôleurs, les styles d'API, le moteur de listes, le moteur de formulaires, les vues et les langues, la sécurité et la documentation d'API.